CS-104 Quiz 3 (ANSWER KEY)

Fall 2026

Name: _____

This is an in-class, closed-book quiz (30 points). If you have any questions, please feel free to ask them. When you are finished, you can turn it in and go.

1. (6 points)

Write a function called `average_above` that receives a list of numbers `values` and a number `limit`, and returns the average of only the elements that are greater than `limit`. If no element is greater than `limit`, it returns `0`. For example, `average_above([3, 8, 1, 9, 5], 4)` returns `7.333...` (the average of 8, 9, and 5). Then write one line that calls your function with that same example and prints the result.

```
def average_above(values, limit):
    total = 0
    count = 0
    for v in values:
        if v > limit:
            total = total + v
            count = count + 1
    if count == 0:
        return 0
    return total / count

print(average_above([3, 8, 1, 9, 5], 4))
```

Grading: header 1, loop with condition 1, sum and count 2, the "none above" case and `return` 1, call and print 1.

2. (3 points)

The code below crashes. What is printed before the crash, what error happens and why, and how would you fix the function?

```
def discount(price):
    new_price = price * 0.9
    print(new_price)

# --- main program ---
final = discount(50) - 2
print(final)
```

It prints `45.0`, then raises a `TypeError`: the function has no `return`, so `discount(50)` gives `None`, and `None - 2` fails. Fix: replace `print(new_price)` with `return new_price`. Grading: 1 point each.

3. (4 points)

Write a function called `cone_volume` that receives the radius (`radius`) and height (`height`) of a cone, and returns its volume. This can be computed as:

volume = (1/3) × pi × radius² × height

where `math.pi` gives the value of pi (assume `import math` has already been done). Be sure to use appropriate Python syntax. (You do not need to provide an algorithm, documentation, or test cases.)

```
def cone_volume(radius, height):
    return (1 / 3) * math.pi * radius ** 2 * height
```

Grading: header with two parameters 1, `math.pi` 1, formula with `**` 1, `return` (not `print`) 1.

4. (3 points)

Using your `cone_volume` function from the previous question, write a code segment that computes the **total** volume of two cones, one with radius 3 and height 4, and one with radius 1 and height 9, and prints the total rounded to 1 decimal place.

```
total = cone_volume(3, 4) + cone_volume(1, 9)
print(round(total, 1))
```

Prints `47.1`. Grading: two correct calls 2, rounding and printing 1.

5. (3 points)

Give the array each expression creates (1 point each). Assume `import numpy as np`.

```
>>> np.arange(4)
```
[0 1 2 3] — example answer

```
>>> np.arange(2, 12, 4)
```

```
>>> np.linspace(0, 2, 5)
```

```
>>> np.arange(0, 1, 0.3)
```

`[2 6 10]`, `[0. 0.5 1. 1.5 2.]`, `[0. 0.3 0.6 0.9]`.

6. (4 points)

A cup of coffee cools down in a room. Its temperature after `t` minutes is

T = 20 + 70 × e^(-0.1 t)

Write a code segment that uses NumPy to create 300 evenly spaced times from 0 to 60 minutes, computes the temperature at each time (use `np.exp`), and uses Matplotlib to plot temperature versus time with the axis labels `Time (min)` and `Temperature (C)` and a title.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 60, 300)
temp = 20 + 70 * np.exp(-0.1 * t)
plt.plot(t, temp)
plt.xlabel("Time (min)")
plt.ylabel("Temperature (C)")
plt.title("Cooling Coffee")
plt.show()
```

Grading: `linspace` 1, temperature formula with `np.exp` on the array 2, plot with labels, title, and `show` 1.

7. (4 points)

What does this code print? Write every line of output.

```
def puzzle(p, q):
    print("check", q)
    if p < q:
        return q - p
    return p * 3

# --- main program ---
x = puzzle(2, 7)
y = puzzle(4, 1)
z = puzzle(q=5, p=5)
print(x, y, z)
```

`check 7`, `check 1`, `check 5`, `5 12 15`. **Grading: 1 point per line.**

8. (3 points)

What does each `print` statement output (1 point each)? In one sentence, explain why the first result is so different from the second.

```
import numpy as np
values = [10, 20]
arr = np.array(values)
print(values * 2)
print(arr * 2)
print(arr + 5)
```

`[10, 20, 10, 20]`, `[20 40]`, `[15 25]`. **Multiplying a list by an integer repeats the list, while arithmetic on a NumPy array is applied to each element. Grading: 2 points for the first two lines with the explanation, 1 point for the third.**